C U R S 5

Context Engineering (nivel 2):

sisteme de regăsire semantică

Ingineria Ai – Aplicații cu agenți inteligenți
Analiza Datelor Complexe, Facultatea de Sociologie și Asistență Socială, Universitatea Babeș Bolyai

CURS 5.

C1 LLM-uri, API-uri - ce construim și cum pornim
C2 Ecosistemul de modele - alegem modelul de bază
C3 Date și corpus - colectare, curățare, metadate
C4 Prompting, adnotare, context engineering
C5 Embeddings, retrieval, RAG
C6 RAG, agenți, LangChain, LangGraph
C7 Agenți orchestrați - LangGraph,metrici,etică
C8 Integrare aplicație - Gradio
C9 Demo final - prezentări și feedback

Tematică
- RAG: diferența dintre regăsire și generare
- embedding-uri ca reprezentări vectoriale ale textelor
- index vectorial: FAISS + metadate
- primele k rezultate și scoruri de similaritate
- verificare umană a rezultatelor recuperate

Activități practice
- Explorăm corpusul cu tipologii discursive
- Alegem un agent / o bulă discursivă
- Curățăm textele slabe și exportăm un JSONL
- Construim un index FAISS pentru bula aleasă
- Testăm retrieval-ul pe un input politic scurt

Livrabile
- notebooks/student_XX/C5_01_explore_typed_corpus.ipynb
- notebooks/student_XX/C5_02_build_vectorstore.ipynb
- data/bubbles/<agent_slug>.jsonl
- assets/vectorstores/<agent_slug>/index.faiss
- assets/vectorstores/<agent_slug>/index.pkl
- scripts/build_vectorstore.py

→ La finalul C5
- Avem un corpus curat pentru fiecare agent
- Avem un vector store FAISS pentru fiecare bulă
- Putem recupera top-k fragmente relevante

De ce nu ajunge căutarea keyword?

Keyword search

Utilă pentru termeni exacți: nume de instituții, partide, persoane. Nu înțelege sinonime, prescurtări, ironie sau reformulări.

Semantic search

Găsește texte apropiate ca sens chiar dacă nu conțin aceleași cuvinte. Compară vectori în spațiul semantic folosind similaritate cosinus.

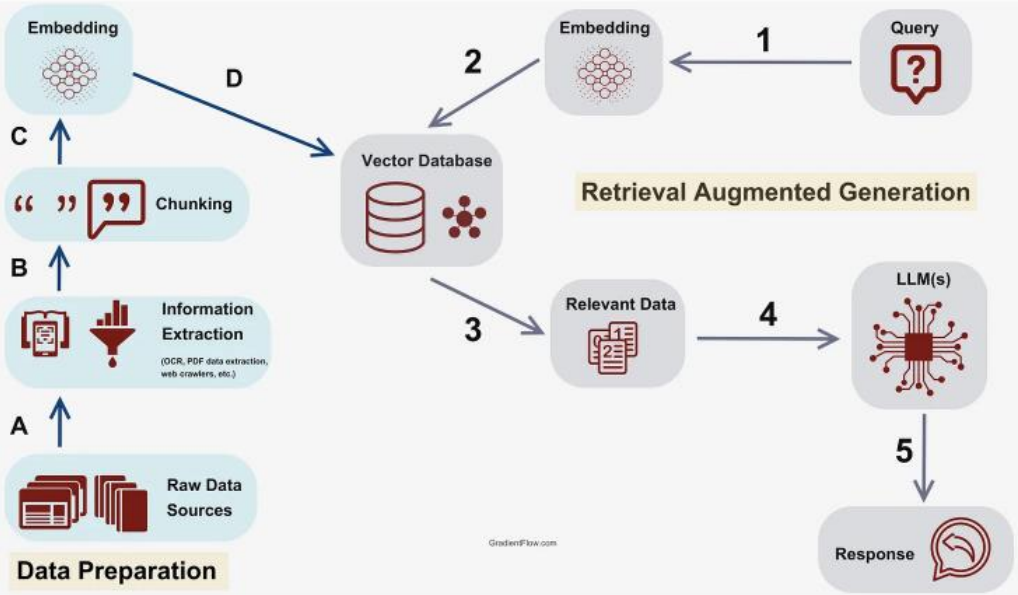
EXEMPLU

„Curtea Constituțională” → *găsește și* ***„CCR”, „judecători constituționali”, „decizie de anulare”***

În producție se folosește adesea hybrid search (keyword + semantic). Weaviate definește hybrid search ca fuziune între vector search și BM25.

Retrieval-Augmented Generation (RAG)

Întrebare → Retriever → Context → LLM → Răspuns



- Modelul nu răspunde doar din ce a învățat la antrenare, ci primește fragmente relevante ca **context**.
- Corpusul este pregătit înainte: texte curate, fragmente, embeddings și vector store.
- La o întrebare nouă, sistemul caută fragmentele cele mai apropiate semantic.
- LLM-ul folosește aceste fragmente pentru a formula un răspuns mai ancorat în date.
- Calitatea răspunsului depinde de calitatea retrieval-ului: date, embeddings, top-k și verificare umană.

https://gradientflow.substack.com/p/best-practices-in-retrieval-augmented

RAG are două jumătăți

C5 — RETRIEVAL

Sistemul caută fragmente relevante din corpus. Embedding-ul query-ului este comparat cu embedding-urile din FAISS. Rezultatele sunt returnate cu scor de similaritate.

Corpus → Embeddings → FAISS → Top-k → Context

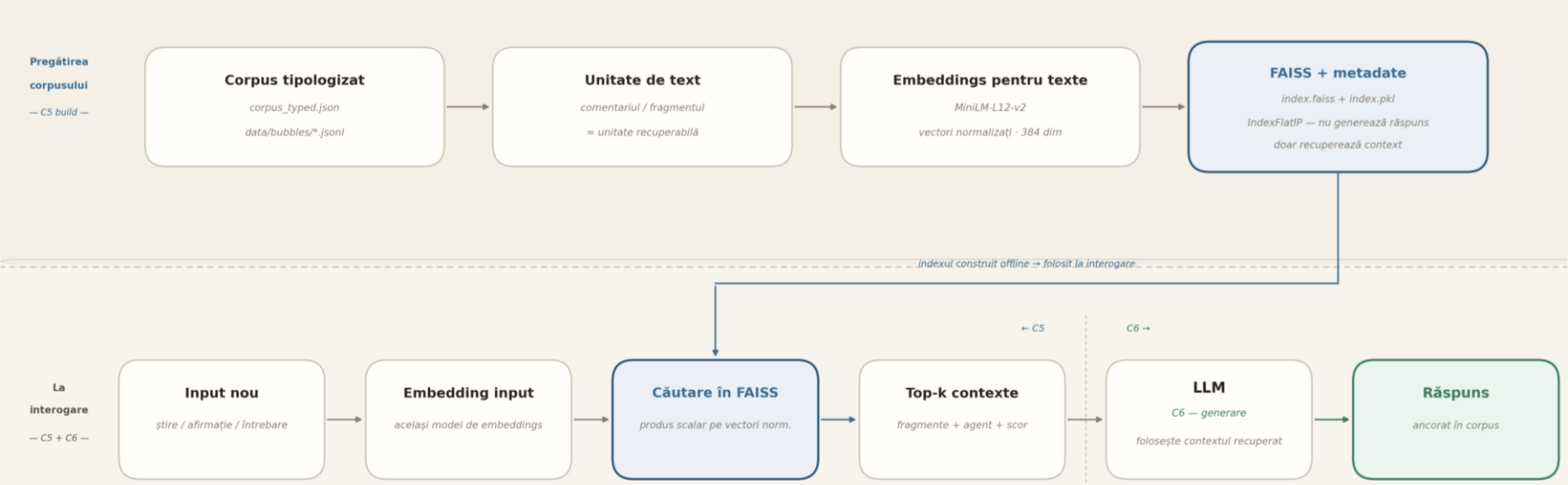
C6 — GENERATION

LLM-ul primește contextul recuperat în prompt și generează răspunsul. Aceasta este etapa în care apar instrucțiunile de rol, restricțiile și formatul de output.

Context → Prompt → LLM → Răspuns

Astăzi construim prima jumătate: cum este ales contextul, nu cum este generat răspunsul.

EchoChamber - RAG

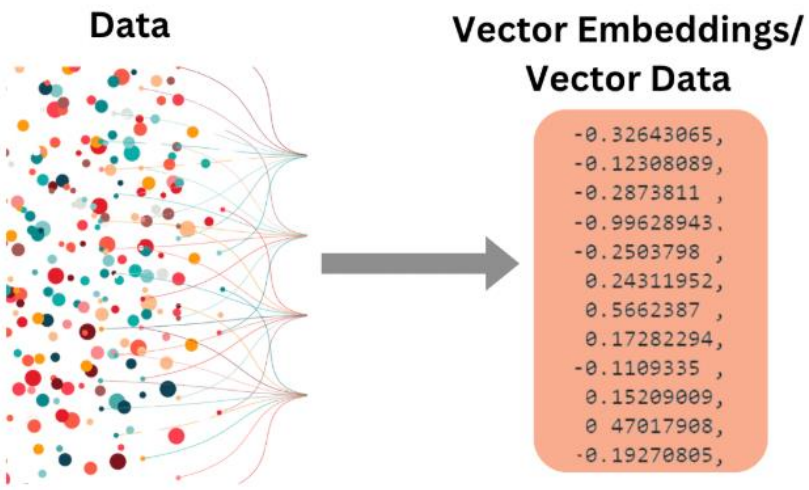


Ce este un depozit vectorial (vector store) ?

Embedding-ul (reprezentarea vectorială) este forma numerică a unui text.

Modelul transformă fiecare comentariu într-o listă de numere, iar texte apropiate ca sens ajung aproape unele de altele în spațiul vectorial.

Depozitul vectorial folosește această geometrie: când primește un input nou, îi calculează embedding-ul și caută textele cu vectorii cei mai apropiați.



https://www.linkedin.com/pulse/complete-guide-creating-storing-vector-embeddings-pavan-belagatti-5fyfc/

Text curat → embedding → vector store
Input nou → embedding → căutare semantică → top-k fragmente relevante

Vector store sau bază de date vectorială?

Text curat
comentariu sau fragment

↓

Embedding
model: MiniLM-L12-v2

↓

Vector (384 dimensiuni)
reprezentare numerică a sensului

↓

Vector store / index
index.faiss + index.pkl

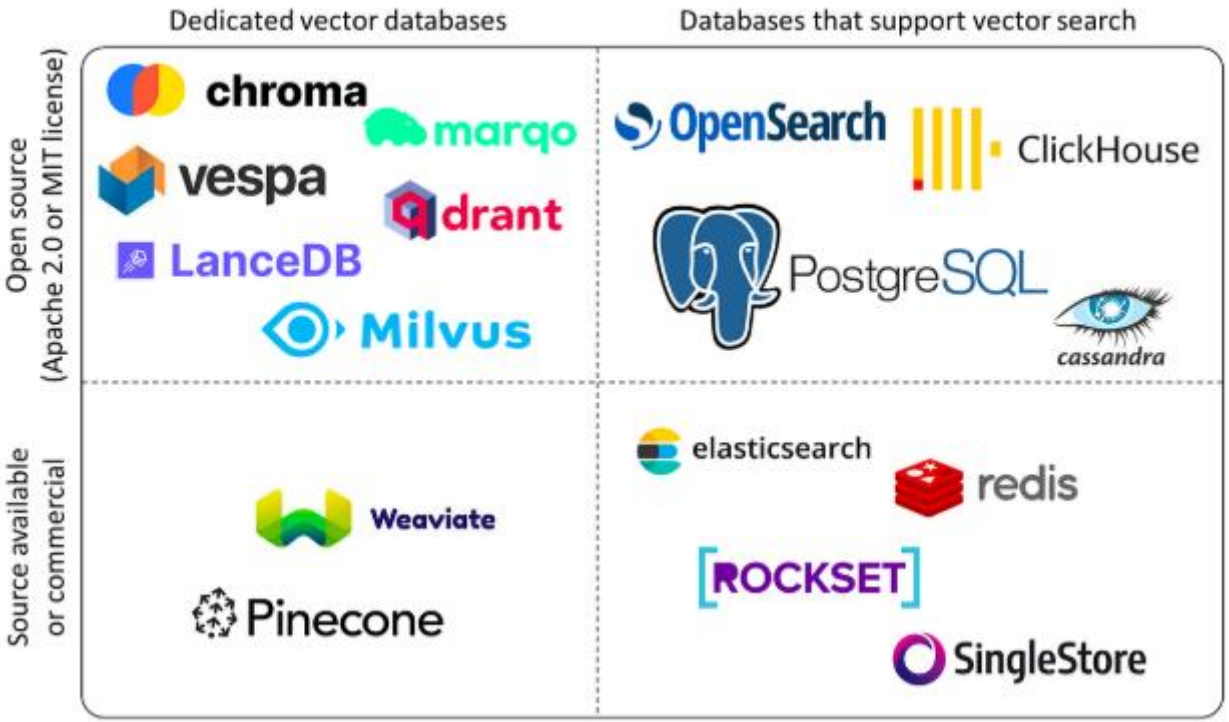
↓

Top-k fragmente similare
fragmente + agent + scor

Caracteristică	Vector store	Vector database
Metadate	separate (ex: index.pkl)	integrate nativ
Filtrare	limitată	filtre + interogări complexe
Persistență	minimă / în memorie	durabilă și sigură
Scalare	dataset mic–mediu	dataset mare / enterprise
Exemple	FAISS, Chroma, LlamaIndex	Pinecone, Weaviate, Supabase/pgvector, Qdrant
Potrivit pentru	prototip, laborator (C5)	producție, aplicații mari

Un **vector store** stochează embeddings și caută texte similare. În C5 folosim **FAISS**, cu metadatele salvate separat în `index.pkl`.
O **bază de date vectorială** face același lucru, dar adaugă filtre, update-uri, securitate și scalare pentru aplicații mai mari.

Baze de date vectoriale



Vector search: de la FAISS la baze de date vectoriale

Toate aceste instrumente ajută la stocarea și căutarea reprezentărilor vectoriale, dar nu sunt același tip de sistem. FAISS este un index vectorial local, potrivit pentru laborator. Chroma este un depozit vectorial persistent. Pinecone, Weaviate și Supabase/pgvector sunt mai apropiate de baze de date vectoriale pentru aplicații mai mari.

Instrument	Ce este	Avantaj principal	Filtrare metadate
FAISS	index vectorial local	rapid, gratuit, transparent	manuală (`index.pkl`)
Chroma	depozit vectorial persistent	stochează texte, vectori și metadate	nativă
Pinecone	bază de date vectorială în cloud	scalare și administrare automată	nativă
Weaviate	bază de date vectorială / motor de căutare semantică	căutare hibridă și metadate	nativă + cuvinte-cheie
Supabase/pgvector	PostgreSQL cu suport vectorial	SQL + vectori + metadate	prin `WHERE` SQL

În C5 folosim FAISS ca index vectorial local. În aplicații mari, aceeași logică poate fi mutată într-o bază de date vectorială.

FAISS în EchoChamber

- FAISS (Facebook AI Similarity Searcheste o bibliotecă pentru căutare rapidă în colecții mari de vectori.
- FAISS nu „înțelege” singur textele și nu generează răspunsuri. El compară vectori și returnează fragmentele cele mai apropiate de inputul nou.
- În laborator folosim **IndexFlatIP**, varianta simplă: caută exact în toți vectorii folosind produsul scalar.

De aceea avem două fișiere separate:

- **index.faiss**

— vectorii numerici (float32)

- **index.pkl**

— textele originale și metadatele (sursă, bulă, dată)

- **core/retriever.py**

— combină cele două și returnează fragmente interpretibile

```
# pipeline build
data/bubbles/<agent_slug>.jsonl
  → encode(normalize_embeddings=True)
  → faiss.IndexFlatIP(dim)  # produs scalar = cosine
  → index.faiss  +  index.pkl

# query
retriever.search(query, k=5)
  → [{text, agent, score}]
```

IndexFlatIP + normalize_embeddings=True: produsul scalar devine echivalentul similarității cosinus.

Embeddings

- Un embedding transformă text în vector numeric de dimensiune fixă (de ex. 384 de valori float).
- Textele cu sens similar au vectori similari - similaritatea cosinus dintre vectori este ridicată.
- SentenceTransformers antrenează modele să producă embeddings dense, optimizate pentru similarity search.

PROPRIETĂȚI CHEIE

- cosine similarity: −1 (opus) → +1 (identic)
- normalize_embeddings=True → dot product = cosine
- dim 384: MiniLM-L12-v2 (mai rapid decât 768)
- multilingv: aceleași texte, mai multe limbi
- limbă: performanța trebuie verificată empiric pe corpusul românesc

```
from sentence_transformers import
SentenceTransformer

model = SentenceTransformer('paraphrase-
multilingual-MiniLM-L12-v2’)

texts = ["Referendumul a fost anulat.",
"Votul CCR este constituțional."]

embeddings = model.encode(texts,
normalize_embeddings=True)

# embeddings.shape: (2, 384)
# fiecare text = un vector de 384 valori
float32
```

LLM-ul nu vede corpusul. Vede un subset.

CORPUSUL COMPLET

Sute sau mii de fragmente colectate în C4. Nu este eficient și nici metodologic util să trimitem tot corpusul în prompt - context window limitat, cost ridicat, pierdere de precizie.

→

CONTEXT SELECTAT

Câteva fragmente alese de retriever, ordonate printr-un scor de similaritate. Ce nu intră în context nu poate influența răspunsul - modelul nu știe că există.

Implicație metodologică: retriever-ul devine filtrul realității modelului. Un retriever biasat produce un model biasat indiferent de calitatea LLM-ului.

Regăsirea semantică este o decizie metodologică

Instrumentul de măsurare decide ce poate fi observat și cum. În RAG, retriever-ul are un rol similar: decide ce fragmente devin vizibile pentru model.

Index: Anti-sistem

Modelul vede instituțiile ca suspecte sau compromise.

Index: Conspiraționist

Modelul vede evenimentele prin actori ascunși și coordonări externe.

Index: Pro-european

Modelul vede regulile și instituțiile ca repere de stabilitate.

Fluxul de regăsire semantică

Corpus → Chunking → Embeddings → FAISS → Query → Top-k → Context

- Pipeline-ul de retrieval are pași separați, fiecare cu consecințe asupra calității contextului.
- La noi, corpusul este deja împărțit în texte scurte; de aceea chunking-ul este mai degrabă verificarea unității de text decât segmentarea unor documente lungi.

FIECARE PAS = DECIZIE DE CERCETARE

- **chunking**

— unitatea de analiză; granularitatea recuperabilă

- **embedding**

— alegerea spațiului semantic; modelul determină ce înseamnă 'similar'

- **top-k**

— cât din corpus ajunge efectiv în prompt

- **metadata**

— filtru pe sursă, bulă, dată; permite audit și transparență

Embeddings: proximitate, nu adevăr

Embedding-ul transformă textul într-un vector numeric care păstrează relații semantice. Sentence Transformers recomandă normalizarea embeddings pentru dot-product/cosine scoring. Dar apropierea semantică nu garantează adevăr factual, ton corect sau interpretare sociologică validă.

PROBLEME FRECVENTE ÎN DISCURSUL POLITIC ROMÂNESC

- **sarcasm**

— texte ironice pot părea semantic relevante (embeddings nu disting tonul)

- **ambiguitate**

— sensuri multiple se comprimă în același spațiu vectorial

- **bias lingvistic**

— modelele multilingual pot performa diferit pe română; testăm rezultatele pe corpusul nostru și folosim HITL

- **false positives**

— texte semantic similare dar factual greșite sau din contexte diferite

Embedding-ul măsoară proximitate în spațiul vectorial, nu adevăr și nu calitate sociologică.

Laborator C5:

ce construim azi?

NOTEBOOK 1 — EXPLORARE + EXPORT

`notebooks/student_XX/C5_01_explore_typed_corpus.ipynb`

- explorăm corpus_typed.json și alegem agentul final
- verificăm textele, eliminăm exemplele slabe
- exportăm data/bubbles/<agent_slug>.jsonl

NOTEBOOK 2 — BUILD + MINI-TEST

`notebooks/student_XX/C5_02_build_vectorstore.ipynb`

- încărcăm bula curată, generăm embeddings (MiniLM-L12-v2, 384 dim)
- construim IndexFlatIP, salvăm index.faiss + index.pkl
- facem un mini-test de retrieval manual

TEAM SCRIPT

`scripts/build_vectorstore.py`

Construiește vectorstore-uri pentru toate bulele după ce fișierele .jsonl sunt curate.

La finalul C5

La finalul C5, sistemul nu generează încă răspunsuri. El primește un input - o știre, o afirmație sau un context politic - și returnează exemple discursive relevante cu scor. Aceste fragmente devin contextul pe care C6 îl va trimite către LLM.

INPUT

"CCR a decis anularea alegerilor după suspiciuni privind influențe externe."

OUTPUT

```
[
  { text: "...", agent: "Anti-sistem",     score: 0.82 },
  { text: "...", agent: "Conspiraționist", score: 0.77 },
  { text: "...", agent: "Pro-european",    score: 0.71 }
]
```

C5 = caută contextul · C6 = generează cu context · C7 = controlează agenții